

Trade Management Functions

Trade management functions help implement risk control and position sizing:

```
1. def calculate_position_size(account_value, risk_percent, entry_price, stop_price):
2.     # Determine how much money to risk
3.     risk_amount = account_value * (risk_percent / 100)
4.
5.     # Calculate risk per share
6.     risk_per_share = abs(entry_price - stop_price)
7.
8.     # Calculate number of shares to buy/sell
9.     if risk_per_share == 0:
10.         return 0 # Avoid division by zero
11.
12.     shares = int(risk_amount / risk_per_share)
13.
14.     return shares
15.
```

This function implements the common "fixed percentage risk" approach to position sizing used by many professional traders.

*?? **Fun Fact:** During the 2020 COVID-19 market crash, several high-frequency trading firms reported record profits despite (or rather, because of) the extreme volatility. Virtu Financial, for example, saw a 100% increase in trading revenue in Q1 2020 compared to the previous year. A key factor in their success was their modular function-based architecture that could rapidly adjust to changing market conditions. According to industry analysts, firms whose trading systems used well-defined functions with explicit risk parameters were able to adapt within hours to the unprecedented volatility, while more rigidly designed systems had to be temporarily shut down to avoid catastrophic losses. This real-world stress test demonstrated why proper function design isn't just a coding practice—it's a crucial risk management strategy.*